

Design Proposal: Scaling JetStream Stream Sourcing

Status	Draft / Proposal
Area	JetStream — stream sourcing (<code>server/stream.go</code> , <code>server/consumer.go</code>)
Author	source-consumers-linear-scan investigation
Date	2026-06-06
Related	<code>jetstream_sourcing_scaling_test.go</code> , report <code>docs/reports/source-consumers-linear-scan.html</code>

1. Summary

JetStream stream **sourcing** scales poorly in the number of configured sources `S` . Several control-plane operations — consumer setup on leader election/restart, `STREAM.UPDATE` , and `STREAM.INFO` — are **$O(S^2)$** because they map a source *index name* (`iname`) back to its `*StreamSource` config through a linear walk (`stream.streamSource()` , `stream.go:3793`) that is itself called once per source.

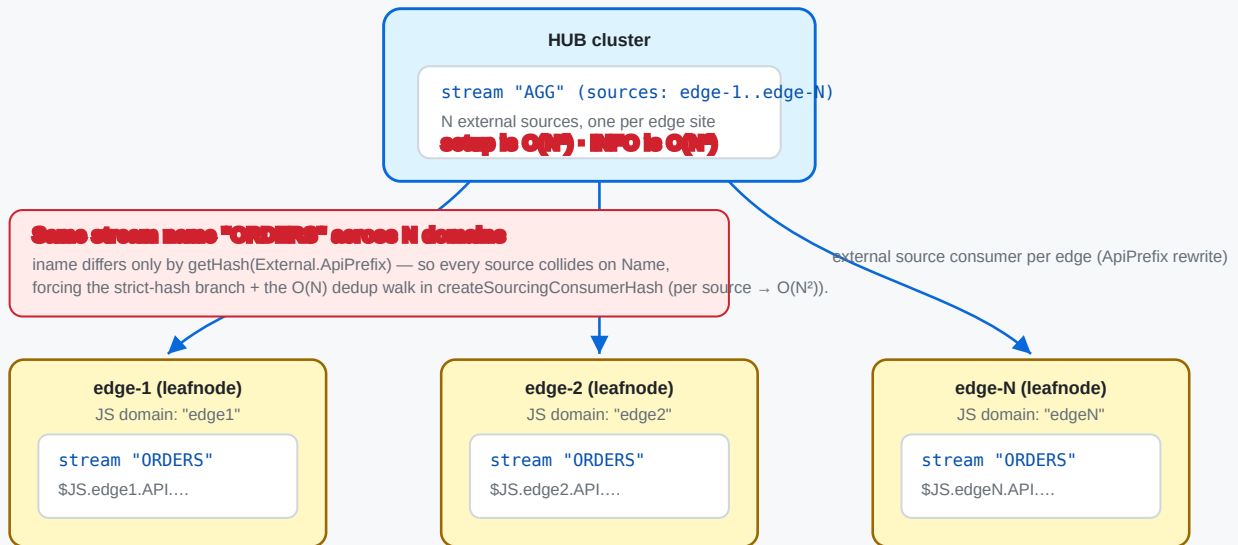
The message **data path is already $O(1)$** and is not changed by this proposal.

We propose maintaining two small derived indexes alongside `cfg.Sources` so that `iname` $\rightarrow$ `config` and `streamName` $\rightarrow$ `sources` become $O(1)$ lookups. This collapses the affected operations from $O(S^2)$ to $O(S)$ with no behavioural change, no wire-format change, and no migration.

2. Motivating scenario

A JetStream instance runs on an **edge leafnode site** with a stream in its own **JS domain**. From the **HUB** side we create a stream that **sources** from that edge leafnode stream. Repeat across many edge sites.

Motivating case: HUB sources from many edge leafnode streams across JS domains



This topology stresses exactly the code paths that scale badly, and adds a second amplifier on top:

1. **Many external sources on one hub stream.** The hub's aggregating stream lists one source per edge site. Each source is *external* (`StreamSource.External` set, with a per-domain `ApiPrefix` such as `$JS.edge1.API.>`). Setting up / refreshing these consumers, and answering `STREAM.INFO0` , all run the $O(S^2)$ paths described in §3.
2. **Stream-name collisions across domains.** Edge sites commonly use the *same* stream name (e.g. `ORDERS`) in different JS domains. The sourcing `iname` distinguishes them only by `getHash(External.ApiPrefix)` (`composeIName` , `stream.go:1079`). Because every source shares the same `Name` , `createSourcingConsumerHash` (`consumer.go:6354`) takes its **strict-hash branch** and performs an $O(S)$ by-name walk — once per source — yielding another independent $O(S^2)$ factor, plus a `getHash(ApiPrefix)` recomputation each time.

So the edge/hub fan-in is the worst-case combination: large `S` , all-external, all name-colliding.

3. Background & problem

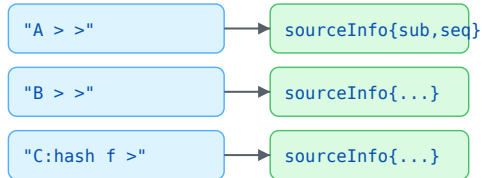
A sourcing stream keeps two halves of state, in two different shapes:

The asymmetry: runtime state is a map, config is a slice

mset.sources

map[string]*sourceInfo

keyed by iname — direct hashed lookup



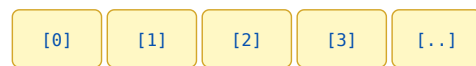
mset.sources[iname] → O(1)

lookup cost independent of source count

mset.cfg.Sources

[]*StreamSource

positional only — no iname index



for _, ssi := range cfg.Sources { if ssi.iname == want {

streamSource(iname) → O(S)

must walk every entry to map iname → config

stream.go:3793

- **Runtime state** lives in a map — `mset.sources map[string]*sourceInfo` (`stream.go:515`), keyed by `iname` , so `mset.sources[iname]` is $O(1)$.
- **Config** lives in a slice — `mset.cfg.Sources []*StreamSource` — with no `iname` index.

Whenever code holds an `iname` and needs the *config* (the external `ApiPrefix` / `DeliverPrefix` , filter, transforms, start policy), it must walk the slice:

```
// stream.go:3793
func (mset *stream) streamSource(iname string) *StreamSource {
    for _, ssi := range mset.cfg.Sources { // O(S)
        if ssi.iname == iname {
            return ssi
        }
    }
    return nil
}
```

That walk is cheap alone, but it is invoked from inside loops that already iterate over every source:

How one walk becomes $O(S^2)$: a loop inside a loop

`setupSourceConsumers()`

runs on leader election / restart / R1 start · stream.go:4805

$S \times O(S) = O(S^2)$

$S=1000 \rightarrow \sim 1,000,000$ string compares

for each of S sources $\rightarrow$ `setupSourceConsumer(iname)`

outer loop · S iterations

`trySetupSourceConsumer(iname)`

stream.go:3897 — needs the *StreamSource config for ext prefix, filters, start policy

`streamSources(iname) — for _, sd := range cfg.Sources { ... }`

inner walk · up to S comparisons · stream.go:3793

`[0][1][2][3] ... [S]`

`[0][1][2][3] ... [S]`

× repeated for every

one of the S sources

All of this happens while holding `mset.mu`, blocking publish

Affected operations

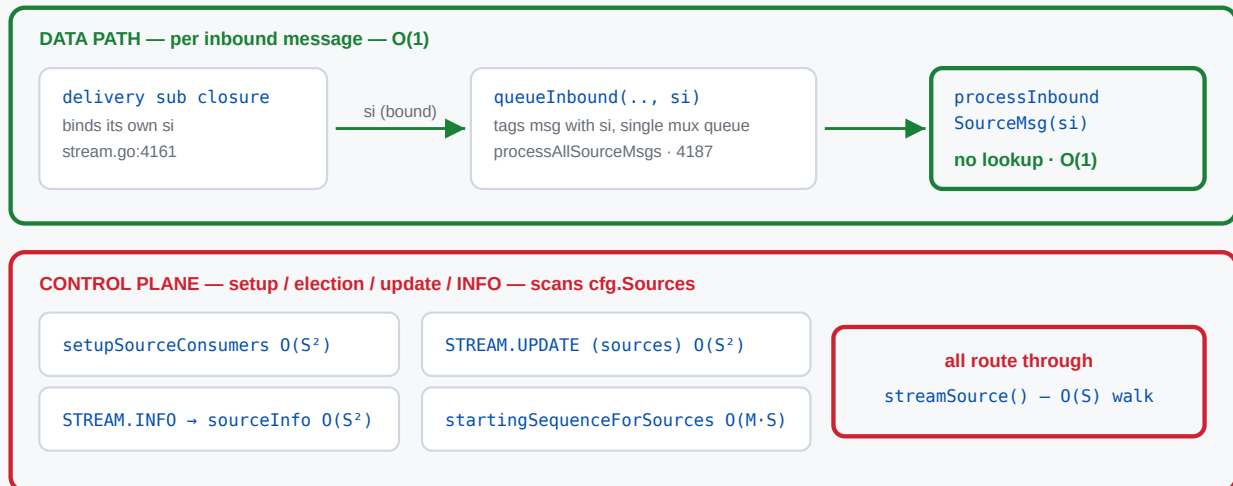
Operation	Trigger	Location	Cost today
<code>setupSourceConsumers</code> $\rightarrow$ <code>trySetupSourceConsumer</code> $\rightarrow$ <code>streamSource</code>	leader election / restart / R1 start	stream.go:4805 , 3911	$O(S^2)$
<code>STREAM.UPDATE</code> source diff (+ <code>getSourcingConsumerIName</code>)	config update	stream.go:2504 , 2509	$O(S^2)$
<code>createSourcingConsumerHash</code> by-name dedup walk	per source in update/setup	consumer.go:6354	$O(S^2)$ when names collide
<code>sourcesInfo</code> $\rightarrow$ <code>sourceInfo</code> $\rightarrow$ <code>streamSource</code>	<code>STREAM.INFO</code>	stream.go:2953 , 2992	$O(S^2)$
<code>setStartingSequenceForSources</code> (the standing // TODO ... linear walk)	config update	stream.go:4566 , 4641	up to $O(M \cdot S^2)$
<code>startingSequenceForSources</code> legacy- header match	setup / restart	stream.go:4694 , 4789	up to $O(M \cdot S)$

M = number of messages scanned backwards through the store; S = number of configured sources.

What is *not* a problem

The hot path stays $O(1)$: each source's delivery subscription closes over its own `si` (`stream.go:4161`) and hands it straight to `processInboundSourceMsg(si, ...)` (`stream.go:4314`). Messages are multiplexed onto one goroutine (`processAllSourceMsgs`, `stream.go:4187`).

The data path is already $O(1)$ — only the control plane scans



4. Goals / Non-goals

Goals

- Make the source control-plane $O(S)$ instead of $O(S^2)$.
- Keep the change behaviour-preserving: no wire format, no config, no on-disk change.
- Specifically remove the cost amplifiers in the edge → hub external/name-colliding case.

Non-goals

- Changing the data-path throughput model or sharding the single source goroutine (`TODD(dlc)` at `stream.go:4187`) — separate, larger work.
- Changing how `iname` / `cname` are composed, or the source header format.

5. Proposal

Derive two indexes from `cfg.Sources`, rebuilt whenever the sources list changes, under `mset.mu`.

Proposal: derive two indexes when `cfg.Sources` changes

`mset.cfg.Sources []*StreamSource`
source of truth (unchanged)
mutated at: load, resetSourceInfo, update

`rebuildSourcesIndex()`
one helper, lock held,
pure derived state

NEW • `cfgSources`
`map[iname]*StreamSource`
`streamSource() → map read · O(1)`

NEW • `cfgSourceNameCount`
`map[streamName]int`
kills by-name dedup walk · O(1)

Effect on the control-plane callers

`setupSourceConsumers`

$O(N) \rightarrow O(1)$

`STREAM.UPDATE (sources)`

$O(N) \rightarrow O(1)$

`sourceInfo / sourcesInfo (INFO)`

$O(N) \rightarrow O(1)$

`createSourcingConsumerHash`

$O(N) \rightarrow O(1)$ dedup

`setStartingSequenceForSources`

drop the TODO walk

`legacy header matching`

`map[name]` lookup

```
// stream struct, near stream.go:515
sources          map[string]*sourceInfo // existing runtime state
cfgSources       map[string]*StreamSource // NEW: iname      -> config  (O(1))
cfgSourceNameCount map[string]int        // NEW: stream name -> count  (O(1))
```

```
// Lock held. Pure derived state – rebuilt on any cfg.Sources mutation.
func (mset *stream) rebuildSourcesIndex() {
    cs := make(map[string]*StreamSource, len(mset.cfg.Sources))
    nc := make(map[string]int, len(mset.cfg.Sources))
    for _, ssi := range mset.cfg.Sources {
        if ssi.iname == _EMPTY_ {
            ssi.setIndexName()
        }
        cs[ssi.iname] = ssi
        nc[ssi.Name]++
    }
    mset.cfgSources, mset.cfgSourceNameCount = cs, nc
}
```

```
// O(1) replacement for the linear walk.
func (mset *stream) streamSource(iname string) *StreamSource {
```

```
    return mset.cfgSources[iname]
}
```

Call sites that must rebuild the index (the only places `cfg.Sources` changes):

- initial config load / `resetSourceInfo` (`stream.go:4659`),
- the `STREAM.UPDATE` source diff where sources are appended/removed (`stream.go:2530` , `2570`).

Tier 1 — iname index (highest impact, smallest change). Replacing `streamSource()` with the map read drops `setupSourceConsumers` , `sourceInfo` / `sourcesInfo` , and the two `streamSource()` calls in `setStartingSequenceForSources` (the `// TODO`) from $O(S^2)/O(S)$ to $O(1)$ each.

Tier 2 — name-count + name index (covers the edge/hub case).

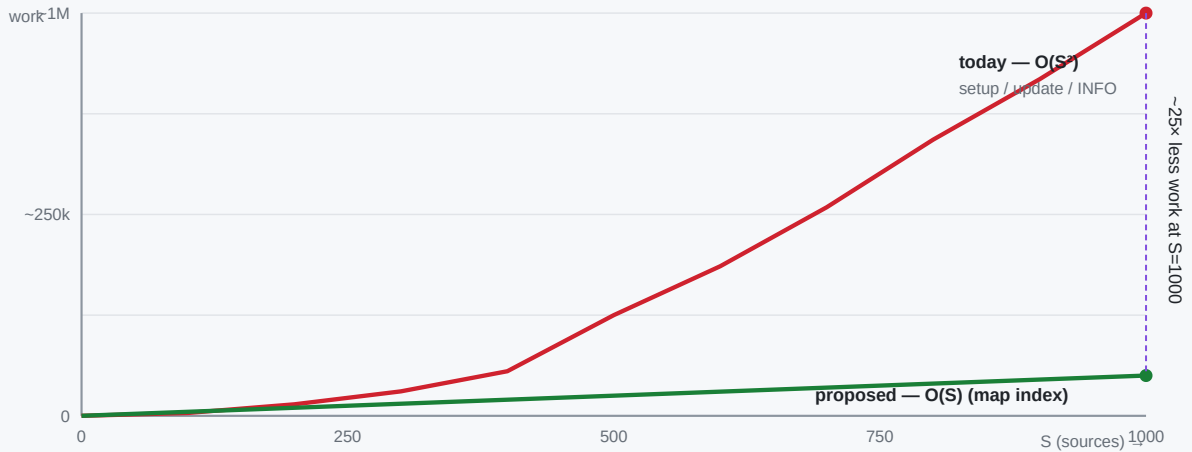
- In `createSourcingConsumerHash` , replace the by-name loop (`consumer.go:6361`) with `if mset.cfgSourceNameCount[ssi.Name] > 1 { ... strict hash ... }` .
- For legacy-header matching (`stream.go:4789` , `4638`), add a `map[streamName][]*StreamSource` (or reuse the count map to gate a fallback) so the inner by-name walk becomes a lookup.
- Optionally memoize `getHash(ApiPrefix)` per source so external `iname` / `cname` /`INFO` construction stops re-hashing.

Tier 3 — validation. Add a `BenchmarkSetupSourceConsumers` over $S \in \{100, 500, 1000\}$ and a down-scaled, *un-skipped* variant of

`TestStreamSourcingScalingSourcingManyBenchmark` (`jetstream_sourcing_scaling_test.go:110`), including an external/domain case. Run with `-race` .

Expected effect

Work per control-plane event vs. number of sources (S)



Operation	Before	After
<code>setupSourceConsumers</code>	$O(S^2)$	$O(S)$
<code>STREAM.UPDATE</code> (sources)	$O(S^2)$	$O(S)$
<code>STREAM.INFO</code> (sourcesInfo)	$O(S^2)$	$O(S)$
<code>createSourcingConsumerHash</code> dedup	$O(S)$ per call	$O(1)$ per call
<code>setStartingSequenceForSources</code> inner	$O(S^2)$ walk	$O(1)$ lookups

6. Alternatives considered

- **Sort `cfg.Sources` and binary-search by `iname`** . $O(\log S)$ not $O(1)$, and re-sorting on every mutation complicates the append-based update path; a map is simpler and faster.
- **Store the `*StreamSource` pointer on `sourceInfo`** . Tempting, but `sourceInfo` is keyed by `iname` already and some callers only have the config slice; a standalone index covers all callers uniformly and keeps the two structs decoupled.
- **Do nothing / document only**. The edge → hub fan-in is a real, growing topology; leaving $O(S^2)$ in place means leader elections and INFO polls degrade visibly as sites are added.

7. Risks & compatibility

- **Index drift.** The derived maps must be rebuilt on every `cfg.Sources` mutation. Mitigation: funnel all mutations through `rebuildSourcesIndex()` and assert `len(cfgSources) == len(cfg.Sources)` in a debug/test build.
- **Uniqueness.** `iname` is already validated unique per stream (`stream.go:1963-1974` , *"check sources for duplicates"*), so the map key is well-defined.
- **Concurrency.** The indexes are read/written only under `mset.mu` , exactly like `mset.sources` .
- **Wire/disk/config:** unchanged. Pure in-memory performance change.

8. Rollout

1. Tier 1 (iname index) as a self-contained, easy-to-review PR + benchmark.
 2. Tier 2 (name index + hash memoization) in a follow-up, motivated by the edge/hub benchmark.
 3. Keep the existing scaling test as the acceptance gate; add the external/domain variant.
-

Appendix — key references

Symbol	File:line	Role
<code>streamSource</code>	<code>stream.go:3793</code>	the linear scan (Tier 1 target)
<code>sources</code> map / <code>cfg.Sources</code> slice	<code>stream.go:515</code>	the asymmetry
<code>composeIName</code> / <code>composeCName</code>	<code>stream.go:1079</code> / <code>1129</code>	iname incl. <code>getHash(ApiPrefix)</code>
<code>setupSourceConsumers</code>	<code>stream.go:4805</code>	$O(S^2)$ setup loop
<code>trySetupSourceConsumer</code>	<code>stream.go:3897</code>	external <code>ApiPrefix</code> rewrite, per-source <code>streamSource</code>
<code>setStartingSequenceForSources</code>	<code>stream.go:4566</code>	TODO-flagged $O(S^2)$
<code>startingSequenceForSources</code>	<code>stream.go:4694</code>	reverse-scan resume
<code>createSourcingConsumerHash</code>	<code>consumer.go:6354</code>	by-name dedup walk (Tier 2 target)

Symbol	File:line	Role
sourceInfo / sourcesInfo	stream.go:2964 / 2953	INFO $O(S^2)$
processInboundSourceMsg	stream.go:4314	$O(1)$ hot path (unchanged)